TML ASSIGNMENT 2 REPORT

Sara Ghanbari
Student ID: 7073225
CMS Team ID: XXXII
sagh00004@stud.uni-saarland.de

Parnian Jahangirad
Student ID: 7062810
CMS Team ID: XXXII
paja00003@stud.uni-saarland.de

1 INTRODUCTION

In this assignment, we had white-box access to one target CIFAR-100 ResNet-18 model and 360 suspect models. Our task was to give each suspect model a continuous stealing confidence score between 0 and 1. Many independently trained CIFAR-100 models agree with the target on easy images, so this gave too many false positives. At the same time, a stolen model may be a direct copy, a fine-tuned checkpoint, a transformed version, or a model trained to imitate the target's outputs. This is also consistent with model extraction and knockoff-model attacks, where the stolen model may copy behavior without sharing weights (Orekondy et al., 2019; Jagielski et al., 2020). For this reason, we treated the problem as model fingerprinting and combined several kinds of evidence.

2 METHOD

Our detector compares each suspect model with the target using fixed probe data and several similarity features. We used multiple feature groups because in early runs we saw that different suspects were suspicious for different reasons. Some models looked close in batch-normalization statistics, while others were only close in output behavior.

Probe data. All models were evaluated on the same probe sets: 2000 target-training member images, 2000 CIFAR-100 test images, 2000 CutMix-style member images, 2000 clean/augmented member pairs, 64 test images for Jacobian comparison, and 1000 non-member training images for dataset-inference features. The seed was fixed to 1337 and the batch size was 256. For the augmentation probe, we used the given target augmentation: reflection padding of 4 pixels, biased crop with bias values 0.5 and -0.25, jitter 0.25, random horizontal flip, and the given CIFAR-100 normalization. CutMix-style probes were added because mixed images can expose model behavior that is less visible on clean examples (Zhang et al., 2018; Yun et al., 2019).

Features. Table 1 lists the main signals used in the final run. Batch-normalization and affine-parameter features mainly target direct copies and lightly modified checkpoints. Output-based features target models that preserve the target's function. Representation, Jacobian, and boundary features add information about internal and local decision behavior.

Feature group	Signal	Main reason for using it
BN / affine statistics	Parameter similarity	Direct copies/checkpoints
Prediction / margins	Output behavior	Functional similarity
SAC variants	Output correlation	Target-specific behavior
CKA / layer cosine	Representations	Similar internal features
Jacobian cosine	Local sensitivity	Similar decision gradients
DDV cosine	Output geometry	ModelDiff-style comparison
Boundary probes	Uncertain regions	Similar near-boundary decisions
Dataset inference	Loss gap	Membership-style trace
Augmentation response	Augmented inputs	Target-specific augmentation behavior

Table 1: Similarity signals used for stolen-model detection.

For output behavior, we used prediction agreement and Spearman correlation of classification margins. We also used sample-correlation fingerprinting (SAC), which compares pairwise relationships

between model outputs instead of only individual predictions (Guan et al., 2022). Besides standard SAC, we used SAC on examples where the target is wrong, following the SAC-w idea of using wrongly classified samples as more target-specific fingerprints (Guan et al., 2022). We also added our own high-confidence SAC variant, which computes the same correlation feature only on samples where the target model is confident.

For representation comparison, we computed linear CKA on final pooled features and cosine similarity on intermediate ResNet activations. CKA is useful here because it compares representation geometry instead of exact neuron matching (Kornblith et al., 2019). We also added a custom Jacobian cosine feature on a small fixed probe set to compare local input-output sensitivity. Finally, we used decision-distance-vector cosine similarity by comparing pairwise distances between output distributions, inspired by ModelDiff (Shah et al., 2023).

The dataset-inference feature is motivated by membership-inference attacks, where differences in model behavior on training and non-training samples can reveal information about the data used to train the model (Shokri et al., 2017).

Scoring. Because the features had different scales, we converted each feature into a robust z-score over the 360 suspects:

$$z_i(f) = \frac{f_i - \text{median}(f)}{1.4826 \cdot \text{MAD}(f) + \epsilon}.$$

We used median and MAD because some features produced extreme values for only a few suspects. Non-finite values were replaced by robust defaults.

We tried BN-only, SAC-only, CKA-only, Jacobian-only, augmentation-only, DDV-only, boundary-only, dataset-inference-only, top-k mean, weighted averages, and max-based scoring. The final submission used

$$s_i = \text{rank}\left(\max_f z_i(f)\right).$$

In our runs, averaging the features often hid the strongest signal. The max rule worked better because it gives a high score when any specialist detector finds strong evidence.

3 Results and Discussion

Our best public leaderboard score was 0.740741 with the `max` variant. BN-only scores were useful for models close to the target checkpoint, but they were not enough for modified or behaviorally copied models. SAC, margin correlation, and DDV helped more for models that preserved the target function rather than its exact parameters, which matches the threat model of functionality stealing (Orekondy et al., 2019; Jagielski et al., 2020).

Representation and local-behavior features were useful as extra evidence. Jacobian and boundary probes were more expensive, but they captured local decision behavior instead of only clean-image predictions. The augmentation-response feature was also useful because the target training augmentation was known. The max rule can also create false positives if one feature is unusually high by chance. We accepted this trade-off because the metric rewards finding stolen models at low FPR, and different stealing methods may only be visible through one or two signals.

4 Conclusion

We treated stolen-model detection as a fingerprinting problem. Our final detector combined several signals and assigned a high score when at least one of them strongly matched the target. This design covered direct copies, fine-tuned models, transformed models, and behaviorally copied models. Our best public leaderboard score was 0.740741. This task matters because training a strong model requires data, computation, and engineering effort. If an attacker can copy the model cheaply, they can benefit from this work without permission. Detecting stolen models is useful for protecting intellectual property, checking model provenance, and making deployed ML systems more trustworthy.

GitHub Repository: `https://github.com/ParnianJR/TML_Assignment2`

REFERENCES

Jiyang Guan, Jian Liang, and Ran He. Are you stealing my model? sample correlation for fingerprinting deep neural networks. In *Advances in Neural Information Processing Systems*, 2022. URL https://arxiv.org/abs/2210.15427.

Matthew Jagielski, Nicholas Carlini, David Berthelot, Alex Kurakin, and Nicolas Papernot. High accuracy and high fidelity extraction of neural networks. In *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 2020. URL https://arxiv.org/abs/1909.01838.

Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 3519–3529, 2019. URL https://arxiv.org/abs/1905.00414.

Tribhuvanesh Orekondy, Bernt Schiele, and Mario Fritz. Knockoff nets: Stealing functionality of black-box models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. URL https://arxiv.org/abs/1812.02766.

Harshay Shah, Sung Min Park, Andrew Ilyas, and Aleksander Madry. ModelDiff: A framework for comparing learning algorithms. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. URL https://arxiv.org/abs/2211.12491.

Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. In *2017 IEEE Symposium on Security and Privacy*, pages 3–18. IEEE, 2017. URL https://arxiv.org/abs/1610.05820.

Sangdoo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. CutMix: Regularization strategy to train strong classifiers with localizable features. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 6023–6032, 2019. URL https://arxiv.org/abs/1905.04899.

Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018. URL https://arxiv.org/abs/1710.09412.